



**INSTITUTO POLITÉCNICO DE TECNOLOGIAS E CIÊNCIAS  
DEPARTAMENTO DE ENGENHARIA E TECNOLOGIAS  
PROGRAMAÇÃO II - 2025/2026 IIº SEMESTRE**

**SISTEMA DE GESTÃO DE INVENTÁRIO**

**GRUPO Nº:19**

**TURMA : EINF4-T3**

**DOCENTE**

**Sílvia António**

**LUANDA, 2026**

**INSTITUTO POLITÉCNICO DE TECNOLOGIAS E CIÊNCIAS**  
**DEPARTAMENTO DE ENGENHARIA E TECNOLOGIAS**

**INTEGRANTES**

| Nome             | Número   |
|------------------|----------|
| Briana Vicente   | 20232111 |
| Delfina Ramos    | 20230596 |
| Elizandra Manuel | 20230009 |

## Índice

|                                                            |           |
|------------------------------------------------------------|-----------|
| <b>1-Introdução .....</b>                                  | <b>4</b>  |
| <b>2-Objectivos .....</b>                                  | <b>5</b>  |
| <b>2.1-Objectivo geral.....</b>                            | <b>5</b>  |
| <b>2.2-Objectivos Específicos.....</b>                     | <b>5</b>  |
| <b>3-Análise de Requisitos .....</b>                       | <b>5</b>  |
| <b>3.1-Requisitos Funcionais .....</b>                     | <b>5</b>  |
| <b>3.2-Requisitos Não-Funcionais .....</b>                 | <b>5</b>  |
| <b>4-Visão Geral da Arquitectura .....</b>                 | <b>5</b>  |
| <b>5-Funcionamento do Sistema e Interface do Menu.....</b> | <b>6</b>  |
| <b>5.1 - Autenticação de Utilizador .....</b>              | <b>6</b>  |
| <b>5.2-Menu Principal .....</b>                            | <b>6</b>  |
| <b>5.2-Submenu: Gestão de Lojas .....</b>                  | <b>7</b>  |
| <b>Submenu: Gestão de Produtos .....</b>                   | <b>8</b>  |
| <b>5.3 Submenu: Registrar Vendas .....</b>                 | <b>8</b>  |
| <b>5.4 -Submenu: Configuração .....</b>                    | <b>9</b>  |
| <b>6-Organização Dos Pacotes .....</b>                     | <b>9</b>  |
| <b>7-Descrição das Classes .....</b>                       | <b>9</b>  |
| <b>1-Classe: Loja.....</b>                                 | <b>9</b>  |
| <b>2 -Classe: Produto.....</b>                             | <b>10</b> |
| <b>3-Classe: Venda .....</b>                               | <b>10</b> |
| <b>4 -Classe: ItemVenda .....</b>                          | <b>11</b> |
| <b>5- Classe: InventarioManager.....</b>                   | <b>11</b> |
| <b>6-Classe: LojaManager .....</b>                         | <b>11</b> |
| <b>7-Classe Usuario .....</b>                              | <b>11</b> |
| <b>9- Estruturas de Dados Que serão Utilizadas.....</b>    | <b>12</b> |
| <b>10-Manipulação de Ficheiros .....</b>                   | <b>12</b> |
| <b>12-Conclusão.....</b>                                   | <b>14</b> |

## **1-Introdução**

O presente documento descreve o planeamento, a arquitectura e o funcionamento de um Sistema de Gestão de Inventário que será desenvolvido em Java, no âmbito da disciplina de Programação II. Este sistema tem como finalidade digitalizar e automatizar o controlo de stock de lojas comerciais, permitindo um acompanhamento preciso e em tempo real das movimentações de produtos.

A gestão de inventário é um dos pilares fundamentais de qualquer negócio que trabalhe com produtos físicos. Em lojas de comércio retalhista, a falta de controlo sobre o stock pode resultar em prejuízos significativos: rupturas de stock que afastam clientes, excesso de mercadoria que imobiliza capital, ou simplesmente a incapacidade de saber, a qualquer momento, o que existe armazenado. Este projecto surge como resposta directa a esse problema.

O sistema permitirá cadastrar múltiplas lojas numa única plataforma, associar a cada loja o seu catálogo de produtos com atributos como nome, código, categoria, preço e quantidade em stock, e registar diariamente as vendas realizadas. A partir desses registos de vendas, o sistema actualiza automaticamente o inventário e disponibiliza relatórios sobre o movimento de cada produto ao longo do dia, semana ou período definido pelo utilizador.

## **2-Objectivos**

### **2.1-Objectivo geral**

Desenvolver um sistema de gestão de inventário em Java.

### **2.2-Objectivos Específicos**

- Modelar as entidades do sistema (Loja, Produto, Venda, ItemVenda) como classes Java com atributos encapsulados e métodos bem definidos.
- Organizar o código-fonte em pacotes temáticos (modelos, serviços, utilitários, interface) para facilitar a manutenção e a legibilidade do projecto.
- Utilizar records Java para representar estruturas de dados imutáveis, nomeadamente os itens individuais de uma venda.
- Implementar a persistência de dados em ficheiros de texto (.txt ou .csv) garantindo que os registos de lojas, produtos e vendas sejam guardados e recuperados entre sessões do programa.
- Aplicar ArrayList para gerir colecções dinâmicas de produtos e vendas, permitindo inserção, remoção e pesquisa eficientes.
- Aplicar Vector em contextos onde a sincronização de acesso aos dados seja necessária, como no registo concorrente de vendas.
- Implementar um menu interactivo no terminal que represente todas as funcionalidades do sistema de forma clara e navegável.
- Permitir o registo de vendas diárias por loja, actualizando o stock dos produtos envolvidos em cada transacção.
- Gerar relatórios de movimentação de inventário por loja, produto ou período de tempo.
- Validar entradas do utilizador para garantir a integridade dos dados inseridos no sistema.

## **3-Análise de Requisitos**

### **3.1-Requisitos Funcionais**

**RF1:** O sistema deve permitir adicionar novas lojas

**RF2:** O sistema deve permitir adicionar produtos a uma loja

**RF3:** O sistema deve registar vendas de produtos

**RF4:** O sistema deve consultar disponibilidade de produtos

**RF5:** O sistema deve gerar relatórios de inventário

**RF6:** O sistema deve persistir dados em ficheiros

**RF7:** O sistema deve exigir autenticação de utilizador (login ou cadastro) para permitir o acesso às funcionalidades de gestão.

### **3.2-Requisitos Não-Funcionais**

- **RNF1:** O sistema deve ser escalável para múltiplas lojas e milhares de produtos
- **RNF2:** O sistema deve responder em menos de 1 segundo para consultas
- **RNF3:** O sistema deve ser robusto e tratar exceções adequadamente

## **4-Visão Geral da Arquitectura**

O sistema segue uma arquitectura em camadas simples, adequada para uma aplicação de terminal em Java. A lógica está dividida em quatro responsabilidades distintas: os modelos de dados (entidades), os serviços (lógica de negócio), os utilitários (ficheiros e validação) e a

interface com o utilizador (menu). Esta separação facilita a manutenção do código e torna cada parte independente e testável.

A interacção entre as camadas será unidireccional: o menu chama os serviços, os serviços manipulam os modelos e recorrem aos utilitários para guardar ou carregar dados em ficheiros. O utilizador interage exclusivamente através do menu, nunca acedendo directamente às classes de modelo.

## **5-Funcionamento do Sistema e Interface do Menu**

Ao iniciar o programa, o utilizador terá acesso a sessão de autenticação ,posteriormente ao menu principal do sistema, onde poderá escolher uma das áreas disponíveis: gestão de lojas, gestão de produtos, registo de vendas, relatórios ou configurações do sistema.

### **5.1 - Autenticação de Utilizador**

Antes de aceder às funcionalidades do sistema, o utilizador deve realizar o login ou efetuar um novo cadastro. Esta camada garante que apenas utilizadores registados possam gerir lojas, produtos e vendas. O sistema validará o formato da credencial (email ou telefone) e a complexidade da palavra-passe.

### **5.2-Menu Principal**

#### **Explicação do Funcionamento**

O menu principal representa o centro de controlo do sistema.

Cada opção conduz o utilizador para um submenu específico, responsável por diferentes operações do inventário.

#### **1. Gestão de Lojas**

Esta funcionalidade permitirá administrar todas as lojas cadastradas no sistema. O utilizador poderá:

- Adicionar novas lojas;
- Listar todas as lojas existentes;
- Selecionar uma loja activa;
- Consultar os detalhes da loja seleccionada.

O objectivo deste módulo é organizar o sistema por lojas, permitindo que cada uma possua os seus próprios produtos e vendas.

(Adicionar imagem do submenu Gestão de Lojas)

#### **2. Gestão de Produtos**

Este módulo será responsável pelo controlo do stock e dos produtos de cada loja.

Através desta opção, o utilizador poderá:

- Adicionar produtos à loja;
- Consultar produtos existentes;
- Verificar quantidades em stock;
- Aumentar a quantidade disponível;
- Identificar produtos com stock baixo.

Sempre que um produto for adicionado ou actualizado, os dados serão automaticamente guardados nos ficheiros do sistema.

#### **3. Registo de Vendas**

A funcionalidade de vendas permitirá efectuar transacções comerciais e actualizar automaticamente o inventário.

Neste módulo será possível:

Iniciar uma nova venda;

Consultar o histórico de vendas;

Visualizar o valor total vendido.

Durante uma venda, o sistema verificará se existe stock suficiente antes de concluir a operação. Após a confirmação, o stock será reduzido automaticamente.

#### **4. Relatórios e Consultas**

Este módulo permitirá gerar informações importantes sobre o funcionamento do sistema.

Os relatórios disponíveis incluem:

Relatório da loja actual;

Inventário completo;

Relatório geral do sistema.

Esses relatórios ajudarão no controlo administrativo e na análise das movimentações de produtos e vendas.

#### **5. Configuração do Sistema**

A área de configuração será utilizada para operações administrativas e manutenção do sistema.

Entre as funcionalidades previstas estão:

Visualizar informações do sistema;

Criar cópias de segurança (backup);

Inserir dados de teste.

O backup permitirá proteger os dados armazenados contra perdas acidentais.

##### **5.2-Submenu: Gestão de Lojas**

O submenu “Gestão de Lojas” será responsável pelo controlo das lojas registadas no sistema.

Através deste menu, o utilizador poderá criar, consultar e seleccionar lojas para trabalhar.

Funcionalidades do submenu

##### **1. Adicionar Loja**

Esta opção permitirá cadastrar uma nova loja no sistema. O utilizador deverá introduzir informações como:

ID da loja;

Nome da loja;

Morada;

Número de telefone.

Após a validação dos dados, o sistema criará um novo objecto da classe Loja e armazenará as informações nos ficheiros de persistência.

##### **2. Listar Todas as Lojas**

Mostrará todas as lojas existentes no sistema, permitindo ao utilizador visualizar rapidamente os estabelecimentos cadastrados.

##### **3. Seleccionar Loja**

Permitirá escolher a loja activa que será utilizada nas operações seguintes, como cadastro de produtos e registo de vendas.

##### **4. Ver Detalhes da Loja Activa**

Exibirá informações completas da loja actualmente seleccionada, incluindo:

Nome;

Endereço;

Produtos disponíveis;

Quantidade de vendas realizadas.

### **Submenu: Gestão de Produtos**

Este submenu será responsável pelo controlo do inventário e gestão do stock dos produtos.

#### **Funcionalidades do submenu**

##### **1. Adicionar Produto à Loja**

Permitirá cadastrar novos produtos numa loja seleccionada. O utilizador deverá informar:

Código do produto;

Nome;

Descrição;

Preço;

Quantidade em stock;

Quantidade mínima permitida.

Após o cadastro, o produto ficará disponível para venda.

##### **2. Listar Produtos da Loja**

Mostrará todos os produtos pertencentes à loja activa, juntamente com as respectivas quantidades e preços.

##### **3. Consultar Stock de Produto**

Permitirá verificar a quantidade disponível de um produto específico.

##### **4. Aumentar Stock de Produto**

Será utilizada para repor mercadorias no inventário. O sistema actualizará automaticamente a nova quantidade em stock.

##### **5. Produtos com Stock Baixo**

Apresentará os produtos cuja quantidade esteja abaixo do limite mínimo definido, ajudando no controlo de reposição.

### **5.3 Submenu: Registar Vendas**

Este submenu será responsável pelas operações de venda realizadas pelas lojas.

#### **Funcionalidades do submenu**

##### **1. Iniciar Nova Venda**

Permitirá efectuar uma nova venda. O processo funcionará da seguinte forma:

O utilizador selecciona os produtos;

Define as quantidades desejadas;

O sistema verifica a disponibilidade em stock;

O valor total da venda é calculado automaticamente;

O stock é actualizado após a confirmação.

Além disso, o sistema armazenará o histórico da venda para futuras consultas.

##### **2. Ver Histórico de Vendas**

Exibirá todas as vendas realizadas pela loja activa, incluindo:

Data;

Produtos vendidos;

Quantidades;

Valor total.

##### **3. Valor Total de Vendas**

Mostrará o valor acumulado de todas as vendas registadas no sistema ou na loja seleccionada.

Submenu: Relatórios

O submenu de relatórios permitirá gerar informações detalhadas sobre o funcionamento do sistema.

#### **Funcionalidades do submenu**

##### **1. Relatório da Loja Atual**

Gerará um resumo completo da loja activa, contendo:

- Produtos;
- Quantidade em stock;
- Total de vendas;
- Produtos mais vendidos.

## 2. Inventário Completo

Mostrará todos os produtos existentes no sistema, organizados por loja.

## 3. Relatório do Sistema

Apresentará estatísticas gerais do sistema, como:

Número total de lojas;

Quantidade total de produtos;

Total geral de vendas realizadas.

## 5.4 -Submenu: Configuração

Este submenu conterá funcionalidades administrativas e de manutenção do sistema.

Funcionalidades do submenu

### 1. Informações do Sistema

Mostrará dados gerais sobre a aplicação, como:

Nome do sistema;

Versão;

Disciplina;

Ano lectivo.

### 2. Criar Backup de Dados

Permitirá criar cópias de segurança dos ficheiros do sistema, garantindo protecção contra perda de dados.

### 3. Dados de Teste

Permitirá gerar automaticamente dados fictícios de lojas, produtos e vendas para facilitar os testes da aplicação.

## 6-Organização Dos Pacotes

O projecto será organizado nos seguintes pacotes Java, cada um com uma responsabilidade clara:

**modelo** — contém as classes que representam as entidades do domínio: Loja, Produto, Venda e ItemVenda. São as estruturas de dados centrais do sistema.

**negócio** - contém as classes Inventário manager, LojaManager, UsuarioManager.

— Contém as classes de serviço que implementam a lógica de negócio: LojaServico, ProdutoServico e VendaServico. É aqui que residem as regras de cadastro, actualização de stock e geração de relatórios.

**Persistência** — Responsável por toda a comunicação com o sistema de ficheiros. Contém a classe Ficheiro, que lê e escreve dados em formato de texto estruturado.

**UI** — Contém a classe Menu, responsável pela apresentação das opções ao utilizador e pela leitura das suas escolhas através do terminal.

— Pacote raiz que contém apenas a classe Principal com o método main, ponto de entrada do programa.

## 7-Descrição das Classes

### 1-Classe: Loja

Representará uma loja no sistema

Conterá os seguintes atributos:

- idLoja: String
- nome: String
- endereco: String
- telefone:
- produtos: ArrayList<Produto>
- vendas: ArrayList<Venda>

Métodos Principais

- + Loja(idLoja, nome, morada, telefone)
- + adicionarProduto(Produto): void
- + removerProduto(String idProduto): void
- + consultarProduto(String idProduto): Produto
- + listarProdutos(): ArrayList<Produto>
- + registrarVenda(Venda): void
- + obterStock(String idProduto): int
- + obterNome(): String

## **2 -Classe: Produto**

Representará um produto em stock.

- idProduto: String
- nome: String
- descricao: String
- preco: double
- quantidadeEmStock: int
- quantidadeMinima: int

Métodos Principais

- + Produto(idProduto, nome, descricao, preco, qtde, qtdeMin)
- + aumentarStock(quantidade): void
- + diminuirStock(quantidade): boolean
- + obterQuantidade(): int
- + obterPreco(): double
- + getNome(): String
- + toString(): String

## **3-Classe: Venda**

Será responsável por representar uma transação de venda.

Conterá os seguintes atributos :

- idVenda: String
- dataVenda: LocalDateTime
- itensVenda: ArrayList<ItemVenda>
- totalVenda: double
- idLoja: String

Métodos Principais:

- + Venda(idVenda, dataVenda, idLoja)
- + adicionarItem(ItemVenda): void
- + calcularTotal(): double
- + obterItens(): ArrayList<ItemVenda>
- + obterDataVenda(): LocalDateTime
- + toString(): String

#### **4 -Classe: ItemVenda**

Representar um item individual em uma venda

Atributos

- idProduto: String
- nomeProduto: String
- quantidade: int
- precoUnitario: double
- subtotal: double

Métodos Principais:

- + ItemVenda(idProduto, nomeProduto, quantidade, precoUnitario)
- + calcularSubtotal(): double
- + obterSubtotal(): double
- + toString(): String

#### **5- Classe: InventarioManager**

Terá a função de gerenciar operações de inventário

Conterá os seguintes atributos

- lojas: HashMap<String, Loja>
- todosOsProdutos: ArrayList<Produto>

Métodos Principais:

- + InventarioManager()
- + adicionarLoja(Loja): void
- + obterLoja(String idLoja): Loja
- + listarLojas(): ArrayList<Loja>
- + registrarVenda(String idLoja, ArrayList<ItemVenda>): void
- + consultarStockProduto(String idLoja, String idProduto): int
- + gerarStockBaixo(): ArrayList<Produto>
- + obterRelatorioVendas(String idLoja): String

#### **6-Classe: LojaManager**

Resnponsável por gerenciar operações específicas de loja

- lojaAtual: Loja
- inventarioManager: InventarioManager

Métodos Principais:

- + LojaManager(InventarioManager)
- + selecionarLoja(String idLoja): boolean
- + adicionarProduto(Produto): void
- + obterProduto(String idProduto): Produto
- + listarProdutos(): ArrayList<Produto>
- + obterRelatorioLoja(): String

#### **7-Classe Usuario**

Representa um usuário do sistema, contendo os dados de autenticação como email ou número de telemóvel e senha. Possuirá métodos para acessar e modificar esses dados, além de autenticar o usuário.

#### **8- Classe: GestorFicheiro**

Será responsável por persistir e recuperar dados.

Métodos Principais:

- + carregarLojas(): ArrayList<Loja>
- + salvarLojas(ArrayList<Loja>): void

```
+ carregarVendas(String idLoja): ArrayList<Venda>
+ salvarVendas(String idLoja, ArrayList<Venda>): void
+ verificarIntegridade(): boolean
```

## 9- Estruturas de Dados Que serão Utilizadas

### 9.1- ArrayList

Utilizaremos para armazenar coleções dinâmicas

- Lista de produtos de uma loja
- Lista de vendas
- Lista de itens de uma venda

Vantagem: Acesso rápido por índice, fácil adicionar/remover elementos

### 9.2-HashMap

Uso: Armazenar lojas indexadas por ID

Vantagem: Busca rápida  $O(1)$  por identificador

### 9.3-Vector (opcional)

Uso: Alternativa thread-safe ao ArrayList se necessário sincronização

## 10-Manipulação de Ficheiros

### 10.1 Estrutura de Ficheiros

```
|— lojas.txt          // Lista de lojas
|— loja_001_produtos.txt // Produtos loja 1
|— loja_001_vendas.csv  // Vendas loja 1
|— loja_002_vendas.csv  // Vendas loja 2
```

### 10.2 Formato de Dados (exemplo lojas.txt)

ID\_LOJA|NOME|MORADA|TELEFONE

L001|Loja Centro|Rua Principal, 10|211234567

L002|Loja Norte|Av. da República, 50|218765432

### 3.6.3 Formato de Dados (exemplo vendas.csv)

ID\_VENDA,DATA\_VENDA,ID\_PRODUTO,NOME\_PRODUTO,QUANTIDADE,PRECO\_UNITARIO

V001,2025-01-15T14:30,P001,Arroz 5kg,10,5.99

V001,2025-01-15T14:30,P002,Óleo 1L,5,3.50

## 11- Tecnologias e conceitos que serão aplicados

**Encapsulamento:** Atributos privados com getters/setters

**Herança:** Possibilidade de especializar tipos de lojas

**Polimorfismo:** Diferentes tipos de vendas (venda normal, promoção)

**Abstração:** Interfaces para operações genéricas

- Persistência de Dados
- Leitura/escrita em ficheiros de texto
- Formato CSV para facilitar importação em ferramentas
- Validação de integridade de dados

-Tratamento de Exceções

Para resolvermos o problema de tratamento de exceções utilizaremos :

- Try-catch para operações de ficheiros
- InputMismatchException para entrada de utilizador
- IllegalArgumentException para validações

## **12-Conclusão**

Com o desenvolvimento do Sistema de Gestão de Inventário, será possível aplicar de forma prática os conhecimentos adquiridos na cadeira de Programação II . O projecto permitirá utilizar diversos conceitos importantes da programação, como classes, métodos, ArrayList e manipulação de ficheiros.

O sistema foi pensado para facilitar o controlo de produtos e organização do stock, tornando o processo de gestão mais simples, rápido e eficiente. Além disso, a utilização de ficheiros permitirá guardar as informações dos produtos, evitando a perda de dados.

A realização deste trabalho contribuirá para o desenvolvimento da lógica de programação, da capacidade de resolução de problemas e da compreensão sobre a criação de sistemas informáticos voltados para situações do dia-a-dia.

Endereço do GiyHub: <https://github.com/ElizandraManuel/SistemaDeGest-odeInvent-rio.git>